第1章 编程改变思维

第1节 为什么要学习编程

你是否还在将计算机（电脑）当作上网、聊天和玩游戏的工具？没错，大部分人是这样的。当你拿起本书阅读到这里的时候，太好了，你又多了一个更好的选择，一个独特的机会！

在我们生活的这个时代，你会发现有这样一群人，他们对世界的影响越来越大，电视、报纸和网络到处都充斥着他们的身影。比尔·盖茨创立了微软，让就是就更容易被我们平常人使用；乔布斯创立了苹果，iPhone、iPad和iPod每一样产品都在改变着我们的日常生活；谢尔盖·布林和拉里·佩奇两个年轻的小伙子创立了Google，使得获取知识变得前所未有的容易；马克·扎克伯格创立了Facebook，正在改变人和人之间的关系……甚至12岁的小软件工程师托马斯·苏亚雷斯都在改变我们的世界。他们是这样的一群人？他们为什么会创造奇迹？巧的是他们都有一个共同的特点：在少年时都酷爱计算机编程。计算机编程究竟具有怎样的非凡魔力？计算机变成是否给他们生活带来与常人不同的思维或思考方式？是否是计算机编程为他们开启了不一样的人生道路？

为什么他们从小就开始接触计算机，不但没有沉迷于游戏，反而改变了世界呢？12岁的托马斯说："现在的孩子们不再只是爱玩游戏，他们还想自己制作好玩的游戏，不过孩子们太多不知道去哪里学习计算机编程，而懂得计算机编程的家长又很少。"

其实每个人在童年时期都曾经有个创建游戏的梦想，我们为什么不把这个梦想变成学习的动力呢？大部分孩子都面对计算机的时候缺乏引导，因为他们不知道计算机除了上网、聊天和玩游戏外还能做什么。即使有人想深入地学习计算机，也不知道去哪里学，没有方向，更没有一本简单易懂并且有趣的入门书。

计算机从被发明的那一天起，其使命就是帮助我们提高学习和工作的效率并且改变世界。利用计算机编程，你可以轻松解决数学难题。例如，`__3x6528=3__x8256`，在两个`__`内填入相同的数字使得等式成立。你觉得这样的题目太简单？那么来个稍微复杂点的：`__ __ __ + __ __ __ = __ __ __`，请将1~9这9个阿拉伯数字分别填入9个`__`中使等式成立，每个数字只能使用一次。计算机可以轻轻松松地解决。如果再复杂一点，我想知道上面这个式子的所有解，通过笔算就很困难了，但如果使用计算机编程去解决，就易如反掌，这正是计算机所擅长的。有时你甚至可以利用计算机编程去验证世界性的数学难题，例如，在10000以内去验证哥德巴赫猜想，也都不成立问题。当解决大质数、图论等问题时，计算机编程也是最好的帮手。

那么学计算机究竟是学什么呢？答案是逻辑思维和编程思维。

早在20世界50年代，美国教育界就开始重视计算机编程教学。20世界80年代后，计算机编程教学逐渐进入中小学校，以教程序设计语言为主，目的是提高学生的逻辑推理、判断性思维和动手解决问题的能力。实践证明，学习了计算机编程的中小学生，思考问题的方式变得非常逻辑化，学会了严密的逻辑推理方法，并无形中把他应用到其他学科的学习中。学习计算机编程本质上是在学习一种思维方式——编程思维，它是一种思维体操。青少年本身对计算机有着浓厚的兴趣并且有超强的记忆力，计算机编程将有助于开发其学习潜力，提高逻辑推理能力和解决问题的能力。

学习计算机编程的过程充满乐趣。如果你有一个想法，马上就可以通过编程来实现，并且可以立即看到效果。这种即时的反馈，会让你的学习兴趣变得越来越浓厚，也会让你越来越有信心。这种超强的信心，是你在其他学科中难以感受到的。我还记得我的第一个程序运行成功时的那种兴奋，真是太棒了，你一定要去感受一下，这是你一辈子都不会忘记的感觉。

最后，用笔者的一个武汉二中的学生吕凯风学习编程时的感受结束本小结。

“记得我那时学编程全凭兴趣，兴趣引导我前进。以前做完了作业就打游戏，如今写完了作业就编程。我觉得学会变成后最让自己激动的是，我能用编程来解决几乎所有遇到的数学问题。学数学最强调技巧性，比如7的2000次方除以3的余数是多少？21212157是不是质数？你也许可以用一些小技巧把这两个问题解决掉，但是当我们遇到更难的问题时，往往无能为力。比如214 125 315的123 718 857次方除以12 125 987的余数是多少？2 147 483 647是不是质数？很多实际问题并不像数学中那么理想和美好，许多数学结论，尽管被证明的很巧妙，式子简洁，但是归根到底，如果它只解决了一个特殊问题，则不具有什么实用价值。所以我更喜欢信息学，它告诉我如何去解决一个一般化的问题而不是一个特殊的有技巧型的问题。编程最让我感慨的是它无与伦比的唯一性和严谨性。

说到这里你是不是有点心动了？

第2节 本书是讲什么，写给谁看的

在写本书之前，我反复问了自己几个问题：这本书是讲什么的？这本书是写给谁看的？这本书和一般的编程入门书有什么区别？为什么要写这本书？选用哪一门编程语言入门呢？

第一个问题：这本书是讲什么的？

这是一本编程入门的书。但是本书的重点并不是编程入门，而是向你展示逻辑思维和编程思维的魅力，让你像程序员一样思考。

第二个问题：这本书是写给谁看的？

编程类图书给大众的印象一直是枯燥并且难懂的。究竟什么样的人才能学习编程呢？大学生？高中生？初中生？……不会连小学生都可以吧？！没错，只要有小学四年级的水平，我想你一定可以学习编程，并且轻松读懂本书的全部内容。

如果你的情况恰好符合以下一点或几点，那么本书正是为你所写的。

1、如果你想自己制作好玩的游戏，而不是沉迷于别人的游戏中。

2、如果你曾经对数学感兴趣，我想你一定会喜欢这本书。其实学习编程并不需要精通数学，本书不会出现很复杂的数学公式。即使数学不太好，甚至很糟糕，也完全可以阅读。

3、如果你对逻辑感兴趣，你一定会更喜欢这本书。与计算机展开的逻辑较量一定会让你觉得非常有意思。因为计算机有时会显得不那么听话，你让它朝东可它却偏偏向西，这时候就一定是你编写的程序出现了逻辑问题。而将你的思维清晰有条理地变成计算机程序，就如同一场你与计算机之间的逻辑对决。学习计算机编程能够很好的锻炼你的逻辑思维能力。

4、如果你从小喜欢拆装玩具、改装小电器或是喜欢变废为宝，我想计算机变成会给你带来更多的创造机会。

5、如果你正在读《高中数学必须三》的“算法”一章，那么你一定要阅读本书，去了解计算机编程和算法的本质。

6、正在学习C语言的理工科或者文科的大学生们。

7、准备学习编程的爱好者，或者准备以此为工作的人。

8、哦，对了，还有就是看了很多编程书籍但是一直没有看懂的人，我想你一定可以轻松读懂本书。

9、没有那么多原因，就是喜欢计算机。

第三个问题：为什么写这本书么？

正如前面所说，计算机是一门科学，如果你只是把它当成上网、聊天和玩游戏的工具，那就太可惜了，你将可能失去一个发现自己才能的机会。即使在计算机编程上有一些天赋，你也有可能失去这一机会。和其他人一样，你必须去主动发现自己的天赋和兴趣，就好比你从来没有吃过冰淇淋，就不可能知道自己喜欢吃冰淇淋。然而，如果学习了计算机编程，你就会发现计算机编程如同玩游戏一样有趣，充满活力的思考过程就如同一场比赛令人兴奋。在感受到了计算机编程给你带来的乐趣后，你就再也不会沉迷于计算机游戏，计算机编程将成为你生活中不可缺少的一部分，成为你的一种爱好，成为你的一种学习动力。我想有更多人期待去了解计算机编程。

最后一个问题：选用哪一门编程语言入门呢？

学习编程的重点在于学习编程的逻辑和思维。本书选用较为简单的C语言。你可能要问什么不选C++、C#、Java或者Python之类。因为我觉得相比之下C语言最为单纯，没有那么多连七八糟的东西，非常简洁。即使以C语言为载体，我也尽量做到重点去讲解最有用的内容，而不是C语言的高深语法或者我至今都没有用过的“奇怪”语句。这样就有更多的时间去思考如何解决问题，去关注编程的逻辑和思维。

当然，说到底C语言仅仅是我们与计算机沟通的一门语言而已，相信你在阅读完本书之后，可以很轻松的上手任何一门语言，因为其本质都是一样的。根据TIOBE index的程序语言年度排名，2012年最流行的编程语言正是有着40年历史的C语言，它战胜了Java语言位居TIOBE index榜首，这或许也能说明C语言的重要性吧。以C语言为根本，将帮助你更好地去理解编程的思想，而不仅仅是学会编程。

另外我想告诉你，编程真的是一件非常有趣的事。你就像一个指挥官，让计算机毫无怨言的为你工作。通过编程，你将体会到战胜困难和挑战后的快乐与满足。编程的世界充满无限的可能，只有想不到，没有做不到。当然在编程的时候也会遇到很多问题，我在书中也为你设置了绊脚石，希望你能够顺利的把它找出来。尽信书，不如无书。学习不但要细致，还要有思辨的能力，这样才会有创新，才能总结并创造出自己的东西。现在开始自己动手编程，不要放弃曾经的梦想，大胆的创造你的作品。

艾伦·凯曾经这样说道：“在自然科学中，是大自然都给出一个世界，而我们去套索其中的法则。对于计算机来说，却是我们自己来构建法则，创造一个世界。”

当下，我们的学习不应该在忙碌于重复的计算、记忆等技能。阅读、逻辑推理和主动思考等技能将成为学习的重点。我们应该使用计算机来增强自己的智能，同时发挥人类独有的创造天赋，让我们的思维插上计算机的翅膀。

最后，我保证本书一定不是那种枯燥无味的编程入门书，并且在此之前你一定没有读过如此生动好玩的编程入门书。你可以在茶余饭后阅读本书，或许蹲在马桶上时也可以看得津津有味。现在就让我们一起走进计算机编程的神奇世界，探索和发现计算机编程的魔力吧！

第2章 梦想启航

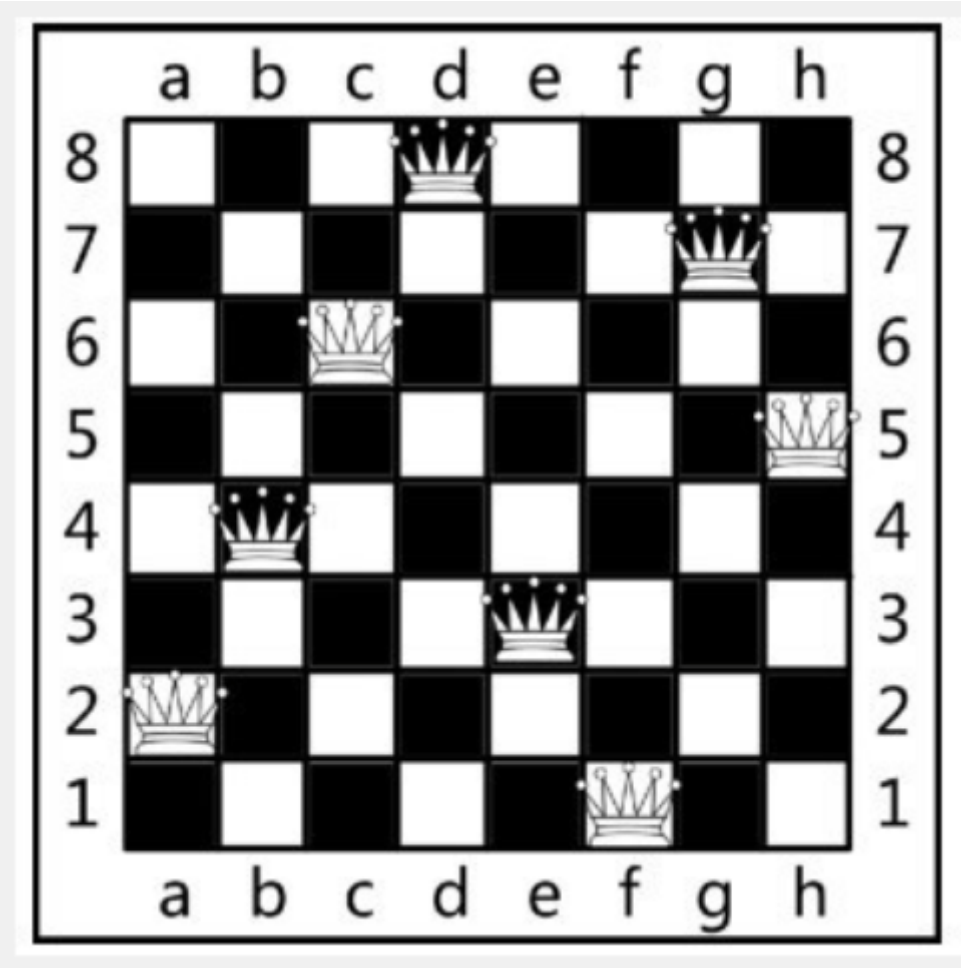
第1节 编程的魔力

从一个神奇的数字说起——2 147 483 647。

2 147 483 647是一个质数（也称为素数，即只能被1和其本身整除的数）。发现这个质数的认识伟大的数学家欧拉。1722年，他在双目失明的情况下，以惊人的毅力靠心算证明了2 147 483 647是一个质数，堪称当时已知的世界上最大的质数，他也因此获得了“数学英雄”的美名。现在你通过计算机只需要1秒钟就可以证明2 147 483 647是一个质数。

再来看一个经典的问题——八皇后问题。

如何能够在8x8的国际象棋棋盘上放置8个“皇后”，使得任何一个“皇后”都无法之际吃掉其他“皇后”？为了达到这个目的，任意两个“皇后”都不能处于同一条横行、纵行或斜线上。下面就是一种解决方案。没错，你可以自己拿出笔和纸画一画，看看还有没有其他方案。但是，如果我想知道所有的方案该怎么办？



又轮到计算机出马了，一共有92种不同的解决方案，很棒吧！计算机只需要1秒，就可以算出所有的解。

再来看一个很流行的益智游戏——数独。

在一个9x9格的大九宫格中有9个3x3的小九宫格，默认在其中填写一些数字，现在请在其它空格上填入1~9。每个数字在每个小九宫格内只能出现一次，每个数字在每行每列也只能出现一次。请看下面这个例子。

	9				2			1
				6				2
						4		
6				8				
	2							
		1	7		4			
3	6							
		7				5		
9	5				7			8

我想，你一定很快就找到了一种可行解，可是你知道上面的这个数独一共有多少种不同解吗？答案是99 410种！很难想象吧，计算机找到这些解仍然只需1秒！怎么样，计算机编程是不是很神奇，你甚至可以轻而易举地在一定范围内去验证“哥德巴赫猜想”。

在接下来的内容里，你将学会如何与计算机对话，如何让计算机进行数学计算和判断，如何让计算机永不停止的工作，以及如何让计算机做一些很有意思的程序和游戏。一场有趣的逻辑思维大战即将开始，不要走开，赶快进入第2节——让计算机开口说话！

第2节 让计算机开口说话

为什么会有计算机的出现呢？伟大的人类发明的每一样东西都是为了帮助我们改善生活。计算机同意是用来帮助我们的工具。想一想，假如你现在希望让计算机帮助你做一件事情，你首先需要做什么？是不是要先于计算机进行沟通？那么沟通就需要依赖于一门语言。人与人的沟通，可以用肢体语言、汉语、英语、法语和德语等。你若要与计算机沟通，就需要使用计算机能够听懂的语言。我们学习的“C语言”便是计算机语言的一种，计算机语言除了C语言之外，还有C++、Java、C#等。C语言是一

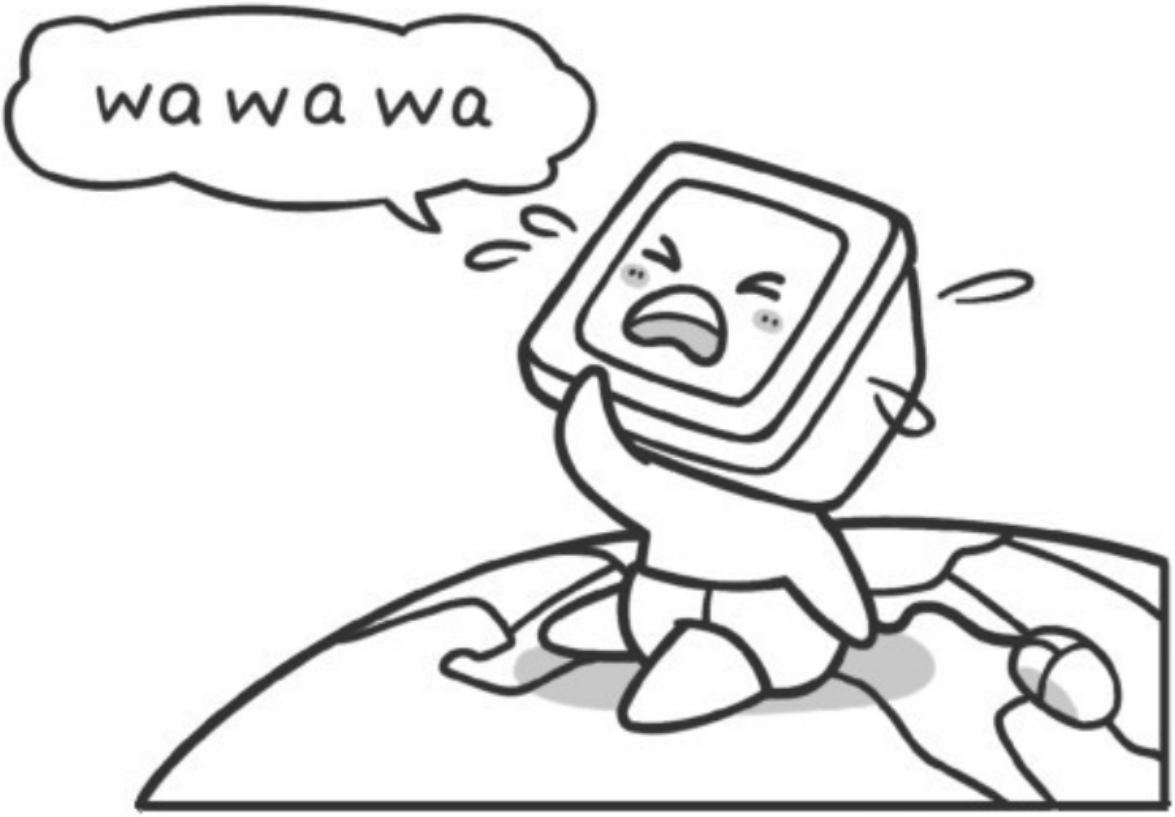
门比较简单的计算机语言，更加适合初学者。所有的计算机语言都是相通的，如果你能够熟练掌握C语言，那么在学习其他语言就会变得易如反掌。

既然计算机是人类制造出来的帮助人类的工具，显然让它开口说话并把它所知道的东西告诉我们是非常重要的。

下面我们就来解决第一个问题：如何让计算机开口说话？

回想当年，我们刚刚开到这个世界的时候，说的第一句话是什么？应该不会是“你好！”、“吃了没？”……这样会把你爸爸妈妈吓坏的。

伴随着“wa wa wa”的一阵哭声，我们来到了这个精彩的世界，现在我们也让计算机来“哭一次”。这个地方特别说一次啊，计算机若要把“它”想说的告诉我们，有两种方式：一种是显示在显示器屏幕上；另一种是通过喇叭发出声音。就如同我们有话想说时，一种是写在纸上，另一种是用嘴巴说出来。目前让计算机用音响输出声音还是比较麻烦，因此采用另一种方法，即用屏幕输出“wa wa wa”。




```
printf("wa wa wa");
```

这里有个生疏单词叫作`printf`，不要被它吓坏了，目前不用搞清楚它的本质意义是什么，只要记住它和中文里面的“说”，以及英文里面的“say”是一个意思即可，它就是控制计算机说话的一个单词而已。在`printf`后面紧跟的`()`，是不是很像一个嘴巴，把要说的内容“放到”这个“嘴巴”里。这里还有一处需要注意，在“wa wa wa”的两边还有“”，里面就是计算机需要“说”的内容，这一点是不是很像我们的汉语？最

后，一句话结束时要有一个结束的符号。汉语中用句号表示一句话的结束；英语中用点好表示一句话的结束；计算机语言中用分号表示一个语句的结束。

注：计算机的每一句话，就是一个语句。

那么，现在如果让你写一个语言，让计算机说“ni hao”，该怎么办？

```
printf("ni hao");
```

现在我们让计算机来运行这个语句。这里需要说明一下，仅仅输入 `printf("ni hao");`，计算机是识别不了的，需要加一个框架。完整的程序如下：

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("ni hao");
    return 0;
}
```

这里的

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    return 0;
}
```

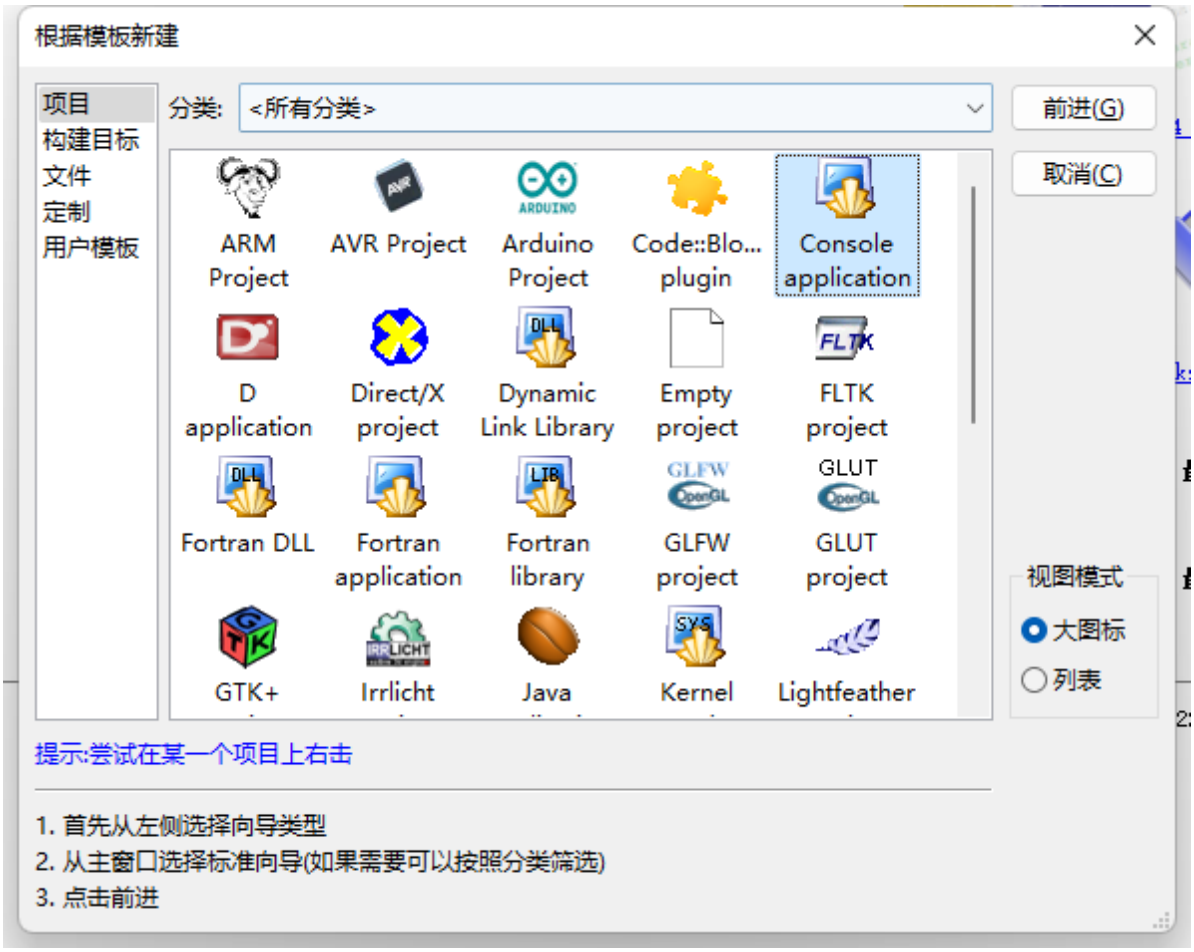
是所有C语言都必须要有的框架，现在暂时不需要理解它，知道要有这个即可，以后再来详细的讲它的用途。但是有一点，我们以后写的所有类似 `printf` 的语句都需要写在 `{}` 里才有效。

接下来我们需要让计算机运行一下我们刚才写的程序。

如果计算机运行我们写的东西（其实我们写的就是一个C语言程序）。需要一个特殊的软件，它叫做“C语言编译器”，C语言编译器有很多种。本期教学种我们使用CodeBlocks，在桌面上找到如下图标。

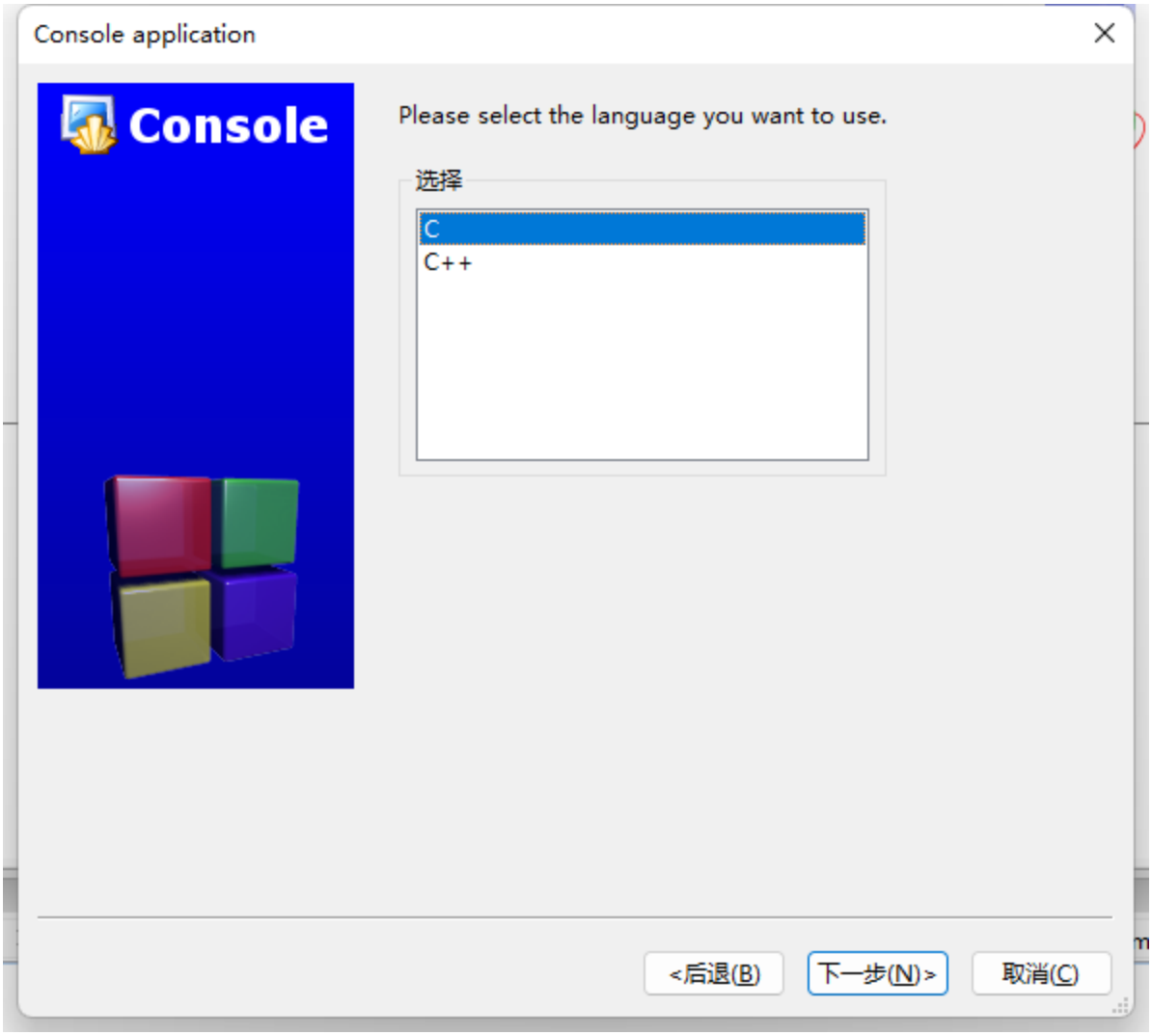
打开界面如下：

点解开始 -> 新建 -> 项目，操作界面如下：

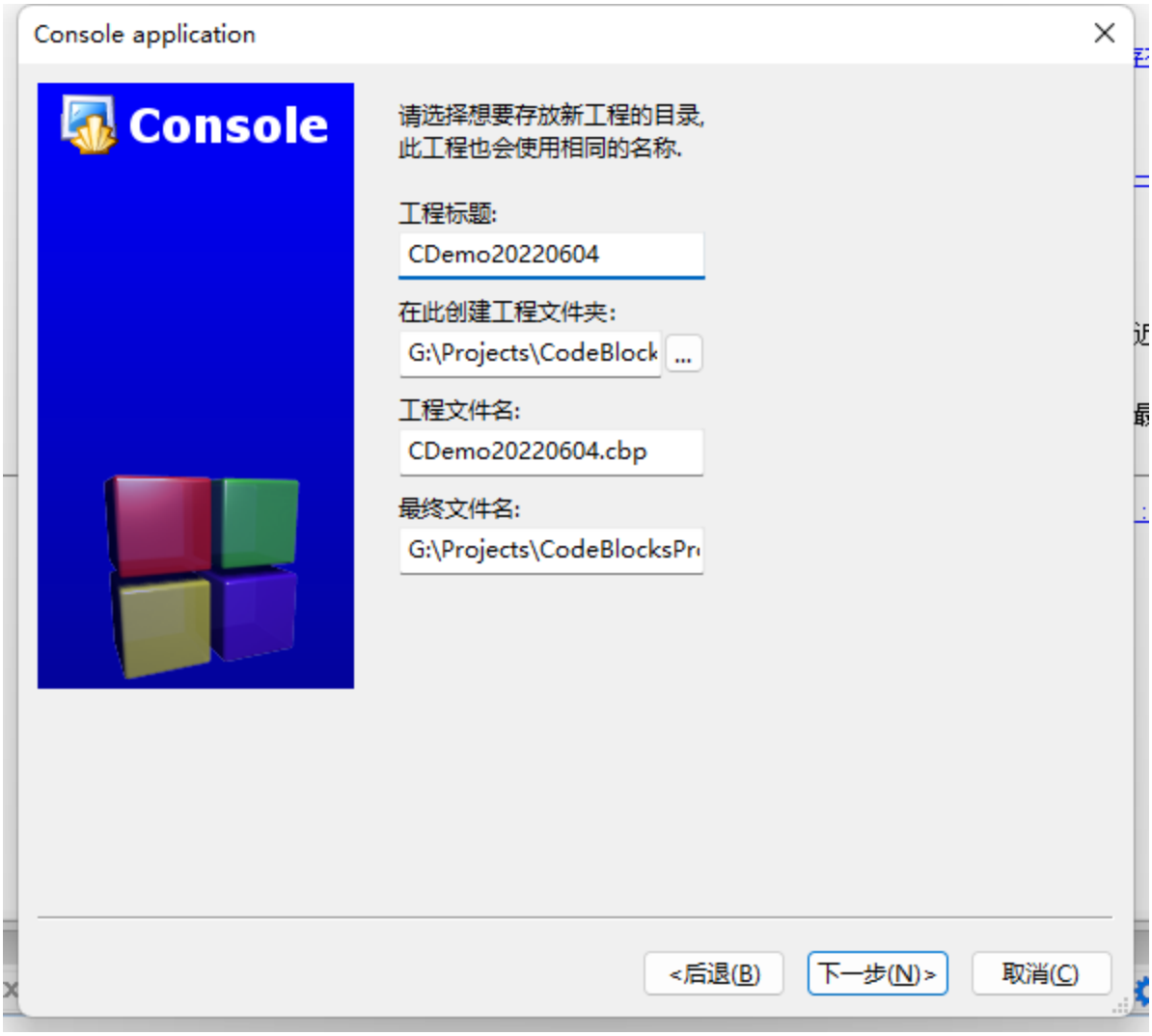


选择 `Console application` 后选在前进进入如下界面：

然后点击下一步，然后选择C语言，如下图

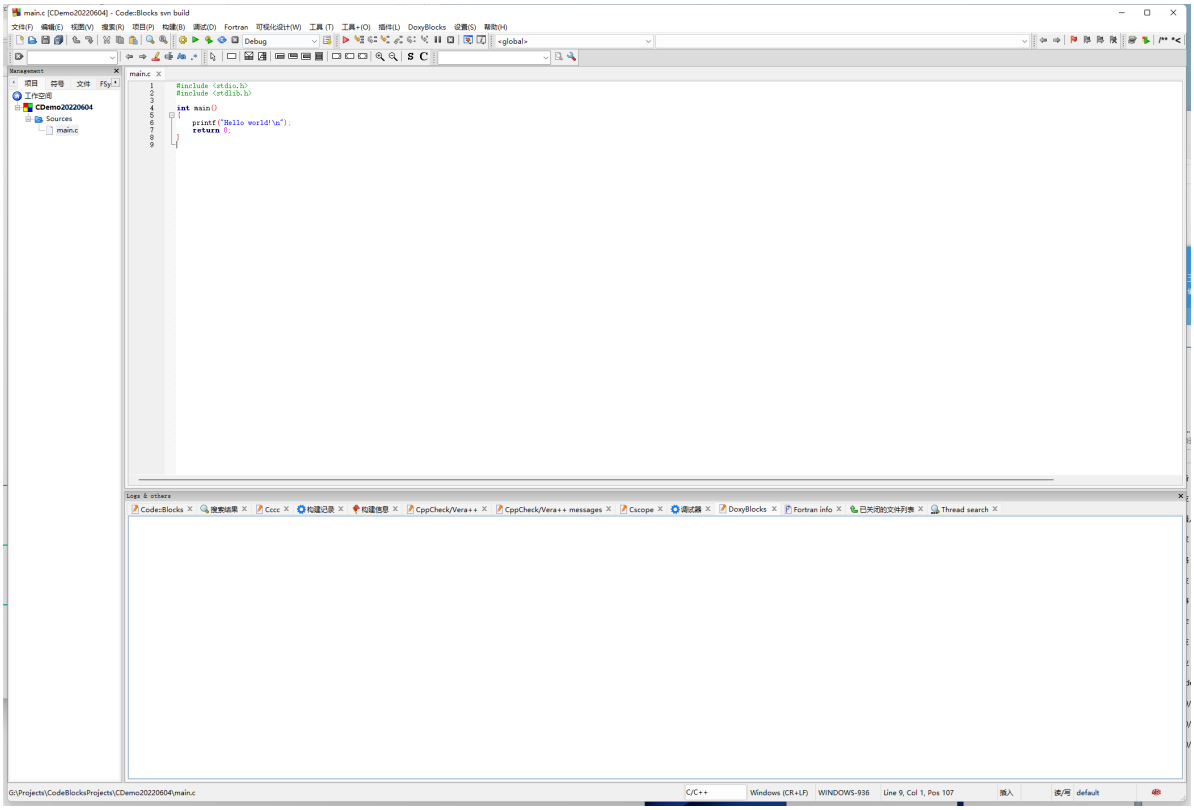


在“工程标题”输入项目名称，在“在此创建工程文件夹”选择项目创建目录。如下界面：

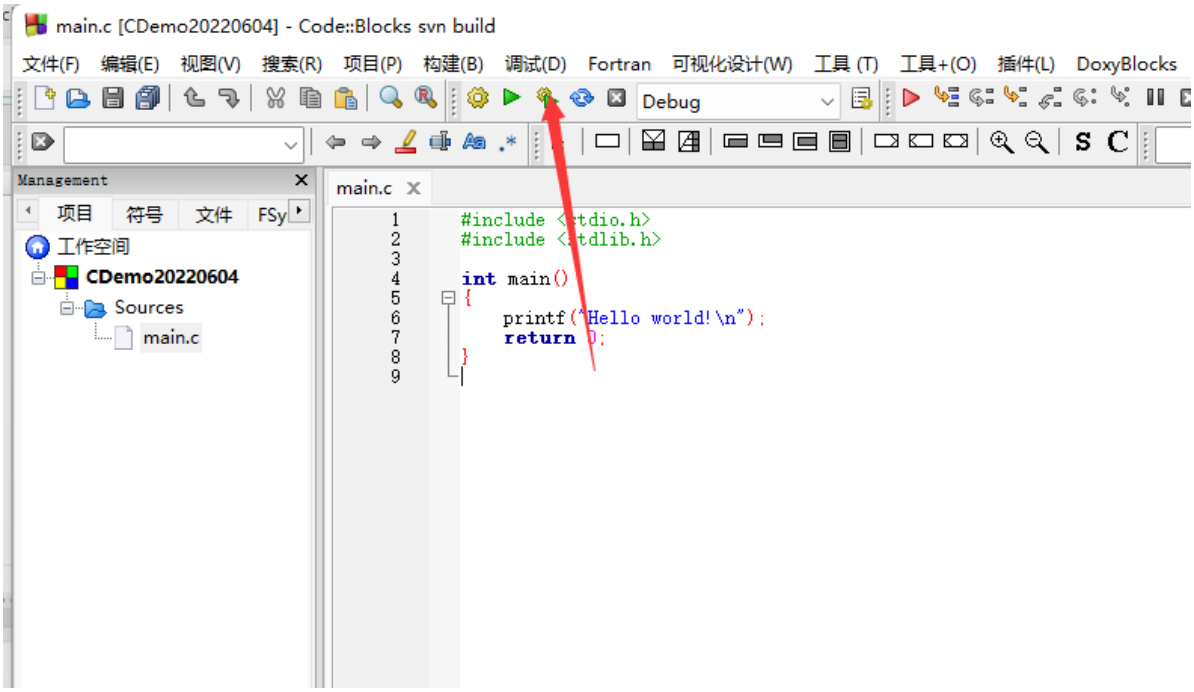


点击下一步，设置编译器版本，默认不需要改变。如下图：

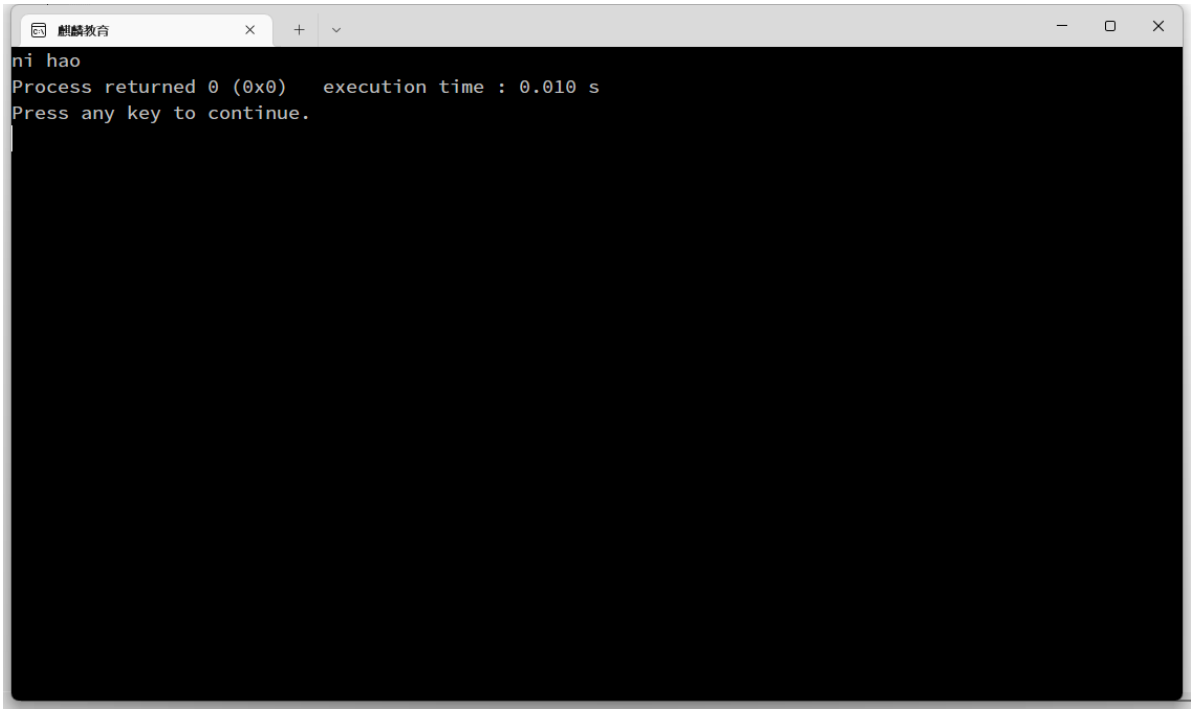
点击完成，完成项目创建，如下图：



C语言代码运行需要先构建生产计算机可以执行的程序，比如windows是.exe文件，Unix是.out文件。构建运营方式如下图：



在上面代码我们把“Hello world！\n”修改为“ni hao”，执行完会显示如下效果：



一起来找茬

1、下面这段代码是让计算机在屏幕上输出“hi”。其中有3个错误，快来改正吧。

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    print(hi)
    sleep(5000);
    return 0;
}
```

更进一步，动手试一试

1、尝试一下让计算机显示下面这些图形吧。

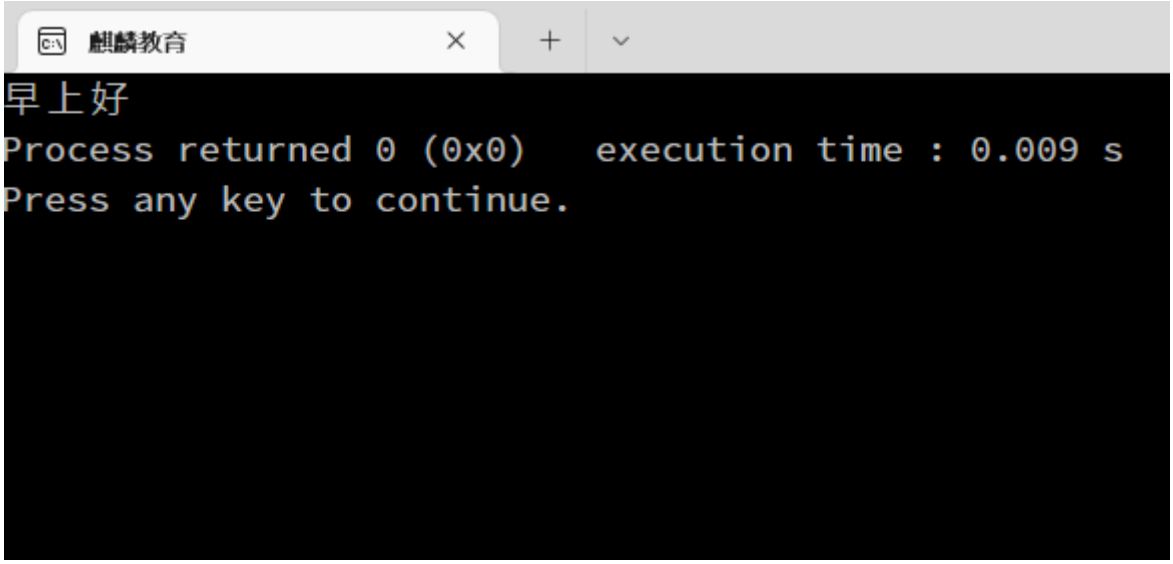
```
*
* *
* * *
```

```
  *
 *  *
*    *
```

```
  *
 *  *
*    *
 *  *
  *
```

```
       *
      *
     *
*   *
 * *
  *
```

2、那么如何让计算机说中文呢？请让计算机说“早上好”，如下图，应该怎么办呢？



3、在尝试一下让计算机显示下面这个图形吧。

```
A
BC
DEF
GHIJ
KLMNO
PRSTUV
W
X
Y
Z
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("*\n**\n***\n");
    getchar();
    system("cls");

    printf("  *\n * *\n*   *\n");
    getchar();
    system("cls");

    printf("  *\n * *\n*   *\n * * *\n  *\n");
    getchar();
    system("cls");

    printf("       *\n      *\n     *\n*   *\n * * *\n  *\n");
    getchar();
    system("cls");
```

```
21
22      printf("早上好\n");
23      getchar();
24      system("cls");
25
26      printf("A\nBC\nDEF\nGHIJ\nKLMNO\nPQRSTUV\nW\nX\nY\nZ\n");
27      getchar();
28      system("cls");
29
30      return 0;
31  }
32
```

这一节，你学到了什么

1、如何让计算机开口说话，让计算机开口说话的语句是？

第3节 多彩一点

在本章第2节中，我们学习了让计算机开口说话应使用`printf`语句。可以发现，计算机“说”出的话都是黑底白字，其实计算机的输出可以是彩色的，我们一起来看看吧。

注意，此处代码只能在Windows操作系统下编译运行。如果你使用的是本书推荐的C语言的软件“CodeBlocks”，那么你的代码肯定可以运行成功。下面来看看如何让颜色出现。

请尝试输入以下代码并运行，看看会发生什么。

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    system("color 5");
    printf("wa wa wa");
    system("pause");
    return 0;
}
```

运行之后你发现了什么？底色仍然是黑色。但是文字的颜色已经变成“紫色”了，奥秘就在下面这行代码中。

```
system("color 5");
```

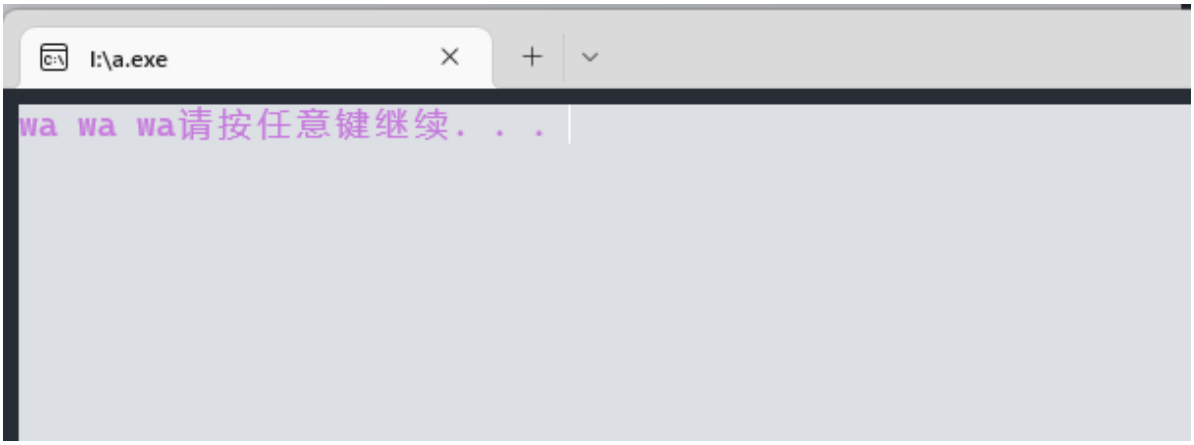
在这句话中，“5”代表“紫色”，你可以尝试一次啊其他数字，看看分别是什么颜色。

既然子的颜色可以变，那么背景色是否可以变呢？尝试下面这段代码：

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    system("color f5");
    printf("wa wa wa");
    system("pause");
    return 0;
}
```

运行成功后的界面如下图所示：



运行成功后的界面（此时北京应该为白色，文字颜色应该为紫色）

上面这段代码在原来的5前面加了一个f，这里的f代表背景色是白色。

那么设置背景色和文字颜色的方法是，在color后面加上两个一位数字，第一个数字表示背景色，第二个数字表示文字颜色。如果在color后面只加了一个一位数字，表示只设置文字颜色，背景色让然使用默认的颜色。

如果说明的是这里的一位数字其实是16进制的，它只能是0、1、2、3、4、5、6、7、8、9、a、b、c、d、e、f中的某一个。

这里学习一个新知识——进制。

在现代数学中，我们通常使用十进制，即使用数字0、1、2、3、4、5、6、7、8、9之后的数字便无法表示了，我们的解决方法是：使用“进制”来表示。例如，由于阿拉伯数字只到9，于是我们便进一位，当前这位用0表示，便产生了用10来表示“十”。因为是“逢十进一”，所以称为十进制。

是十六进制是“逢十六进一”，即使用0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F来表示。0~9与在十进制时相同，但是“十”在十六进制时用大写字母A表示，以此类推，“十五”在十六进制中使用大写字母F来表示。F是“十六进制”中的最后一个，因此数字“十六”就表示不聊了。于是我们有采用刚才在十进制中表示不了就进一位的老方法。当前应该用0表示。“十六”在十六进制中表示为10.同理，“二十七”在十六进制中表示为1B。

在中国古代，很多朝代都是用十六进制作为日常计数的，例如，成语“半斤八两”的典故来源于十六进制；还有中国古代的算法是上面2颗珠子，下面5颗珠子。若上面每颗珠子代表数字5，下面每颗珠子代表1，那么每位的最大计数值是15，15正是十六进制的最大基数。当使用算盘计数遇到大于15的时候，我们就需要在算盘上“进位”了。

其实在我们现代的日常生活中，也不都是“十进制”，例如，60秒为1分钟，60分钟为1小时，就是用的六十进制。

一起来找茬

下面这段代码是让计算机在屏幕上输出绿色白底的hi。其中有4个错误，快来改正吧。

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    system(color f2)
    printf("hi");
    system("pause");
    return 0;
}
```

更进一步，动手试一试

1、尝试让计算机打印这个小飞机图案（绿底白字）。

```
    *
    **
*   ***
**  ****
**** **********
**  ****
*   ***
    **
    *
```

2、尝试让计算机打印这个小对旗图案（白底红字）

```
A
I*
I* *
I* * *
I* * * *
I
I
I
I
I
```

这一节，你学到了什么

让计算机打印出来的字符有不同颜色的语句是什么？

第4节 让计算机做加法

通过之前的学习，我们了解到让计算机说话用`printf`，运用“printf”就可以让计算机想说了什么就说什么了。在学会了“说话”之后，我们看来如何让计算机做数学运算，首先我们先让计算机做“加法”，就先算`1+2`吧。

回想以下小时候爸爸妈妈是如何教我们算`1+2`的呢？

妈妈说：“左手给你一个苹果，右手给你两个苹果，现在一共有几个苹果？”我们迅速地思考了一下，脱口而出：“3个苹果”。没错！我们首先用大脑记住左手有几个苹果，再用大脑记住右手几个苹果，妈妈问一共有几个时，我们的大脑进行了非常快的计算，将刚才记住的两个数进行相加，得到结果，最后将计算出的结果说出来。仔细分析下，大致分为以下5个步骤。

1、用大脑记住左手的苹果数量；

2、用大脑记住右手的苹果数量；

3、我们的大脑将两个数字进行相加；

4、得到结果；

5、将结果输出。

在这期间，我们大脑一共进行了以下4个动作。

1、两次输入：分别是记录左手和右手中苹果的数量；

2、存储了3个值：分别是记录左手和右手中苹果的数量和相加的结果；

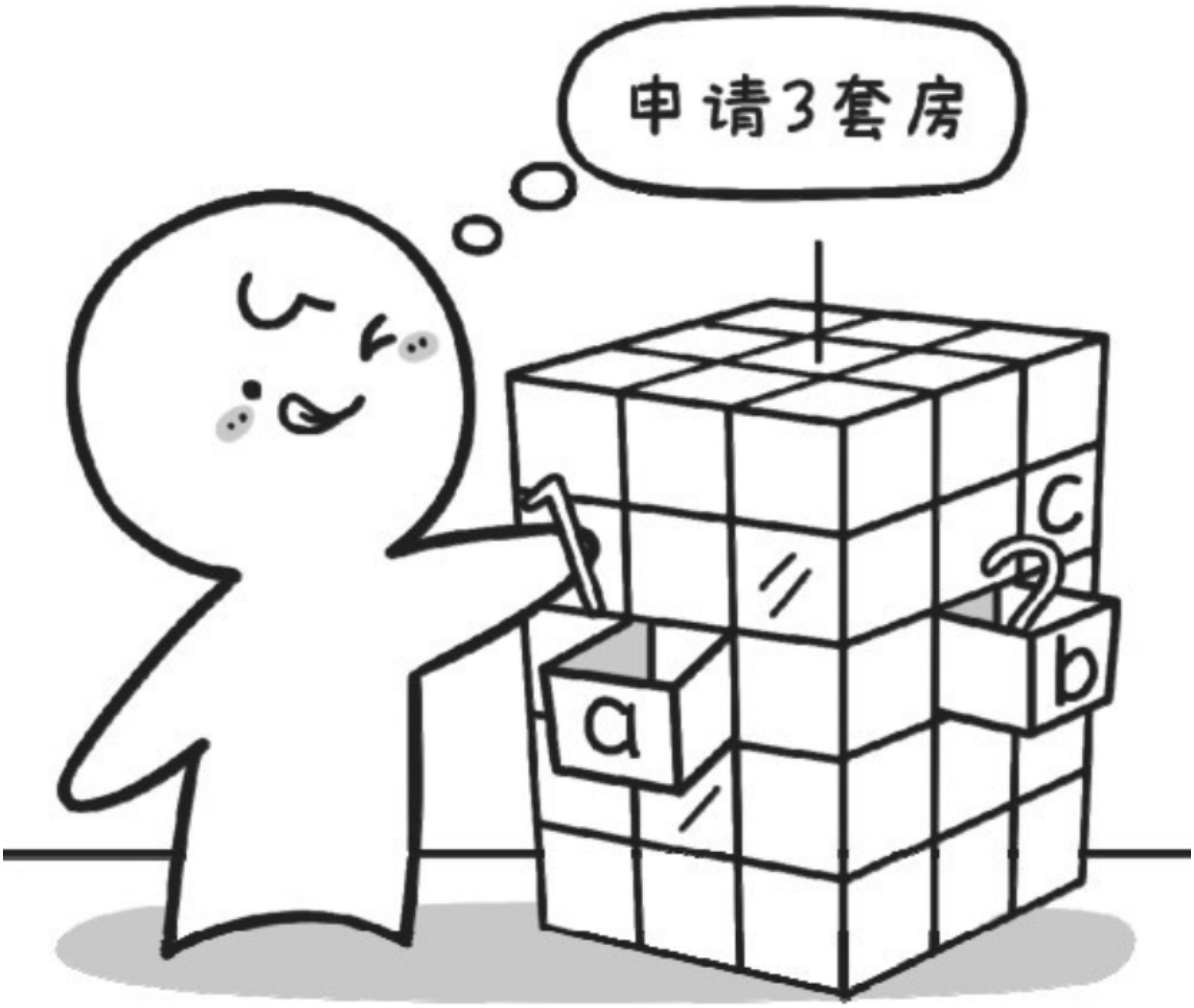
3、进行了一次计算：相加

4、进行了一次输出：吧相加的结果输出。

如何让计算机做加法呢？同样也需要以上几个步骤。

首先来解决如何让计算机像我们的大脑一样记住一个数字。

其实计算机的大脑就像一个“摩天大厦”，有很多一间一间的“小房子”，计算机就吧需要记住的数放在“小房子”里面，一个“小房子”里面只能放一个数，这样计算机就可以记住很多数。好，我们来看一看，具体怎样操作。



“=”赋值符号的作用就相当于一只手，吧数字放到小房子中。

```
int a, b, c;
```

这句话就代表在计算机的“摩天大厦”中申请三件分别叫作a、b和c的小房子（注意：int和a之间有一个空格，a、b和c之间分别用逗号隔开，末尾有一个分号表示结束）。

接下来，我们让小房子a和小房子b分别去记录两个数字1和2，具体如下：

```
a=1;
b=2;
```

说明：此处有一个“=”，这可不是等于号，它叫作给予号（也称为赋值号），类似与一个箭头“⬅“，意思是把“=”右边的内容赋给“=”左边。例如，把1这个数给小房子a，这样一来计算机就知道小房子a里面存储的是数字1了。

然后，把小房子a和小房子b里面的数相加，再将其结果放到小房子c中。

```
c=a+b;
```

计算机会将这个式子分两步执行：第一步先将`a+b`算出来，第二步再将`a+b`的值赋给“=”左边的`c`。

至此，就差不多完成了，我们总结一下：

```
int a, b, c;
a=1;
b=2;
c=a+b;
```

很多同学是不是以为，现在已经全部完成了？你忘记了最重要的一步，先别急着往下看，想一想忘记了什么？

啊~~你忘记了把答案输出。

想一想如果妈妈问你一加二等于多少时，你说：“我算出来了，但是不想告诉你！”的话，估计少不了挨一顿揍了，不要啊！

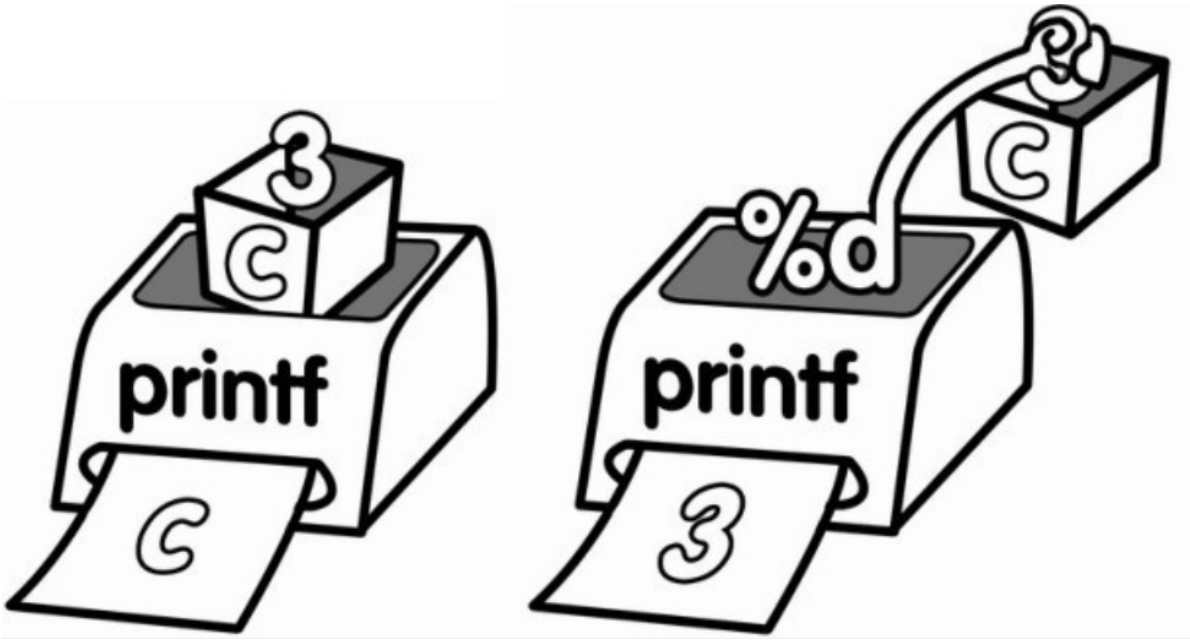
让我们回忆一下，如何让计算机把结果输出。

对，使用`printf`语句。那怎么把小房子c里面存储的数输出呢？根据本章第2节学到的知识，只要把要输出的内容放到双引号里面就可以了，代码如下：

```
printf("c");
```

猜一猜此时计算机会输出什么？

对，无情的输出一个c。



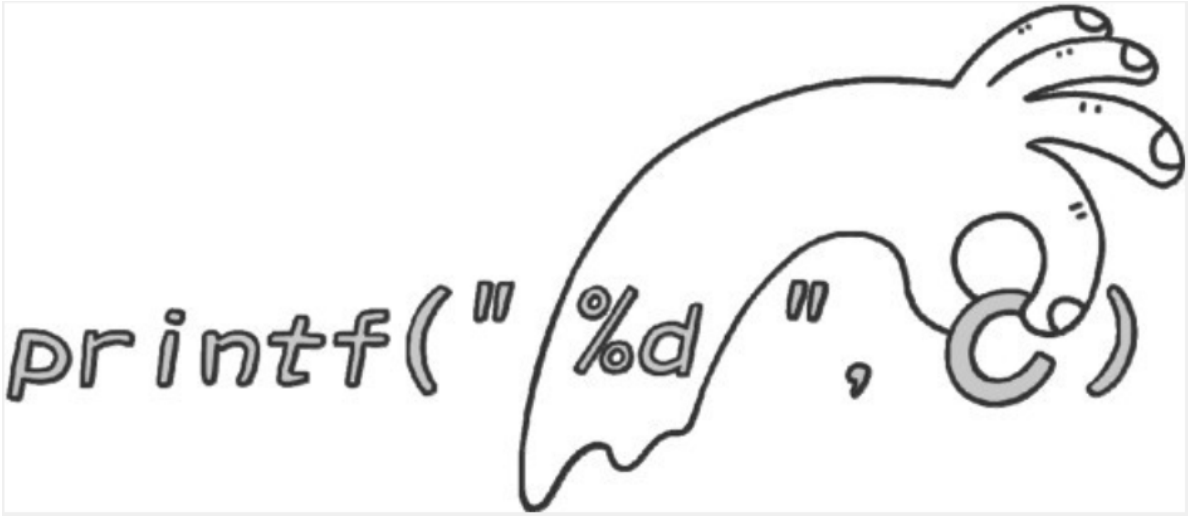
那怎样输出c里面的值呢？

这时我们要让另外一个角色出场了。

```
%d
```

“%d”其实是一个“讨债的”，或者也可以说是“要饭的”。它的专职工作就是向别人“要钱”！那我们应该怎么使用它呢？

```
printf("%d", c);
```



将“%d”放在双引号之间，把小房子c放在双引号后面，并且用逗号隔开。

这时 `printf` 发现双引号里面是个“讨债的：，就知道此时需要输出一个具体的数值，而不是符号，就会向双引号后面的小房子c索取具体的数值了。

好了，最后加上C语言的代码框架，计算机进行加法运算的完整代码如下：

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a, b, c;
    a=1;
    b=2;
    c=a+b;
    printf("%d", c);
    system("pause");
    return 0;
}
```

现在赶紧去试一试吧。

一起来找茬

下面这段代码是让计算机计算 `321-123` 的结果。其中有7个错误，快来改正吧！

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a, b, c;
    a=321
    b=123
    c=b-a
    printf("%d", c)
    system("pause");
    return 0;
}
```

更进一步，动手试一试

1、如果要进行3个数相加的运算，该怎么样做呢？例如：`5+3+1=?`

我们可以把上面的程序进行简单的改变，申请4个小房子分别叫作a、b、c和d。用a、b、c分别存放3个加数，用d存放他们的和。代码如下：

```
#include <stdio.h>
#include <stdlib.h>
```

```

int main()
{
    int a, b, c, d;
    a=5;
    b=3;
    c=1;
    d=a+b+c;
    printf("%d", d);
    system("pause");
    return 0;
}
```

如果要10个数相加岂不是得定义11个小房子？那也太麻烦了~对，目前我们只能这样，但是在后面的学习中，会有更为简单的方法。

2、用计算机算出下面3个算式。

```
123456789+43214321
7078*8712
321*(123456+54321)
```

这一节，你学到了什么

1、如何申请一个小房子和存储数值

2、如何用 `printf` 输出小房子中的数值？

第5节 数字的家——变量

从本章第4节中，我们了解到计算机使用一个个小房子来记住数字。计算机有很多不同类型的小房子，

```
int a;
```

代表像计算机申请一个小房子，用来存放数值，小房子的名字叫作a。int和a之间有一个空格，a的末尾有一个分号，表示这句话结束。

如果要申请多个小房子，则要在a后面继续加上b和c，用逗号分开。例如：

```
int a, b, c;
```

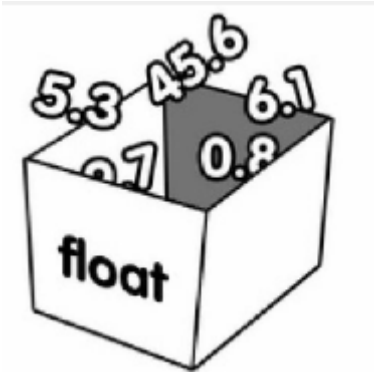
这里有一个小问题，就是给申请的“小房子”起名字。原则上，你可以随意起，叫作a可以，叫作b也可以，叫作x也可以，名字甚至可以是多个字母组合，例如，可以叫作 `aaa`，也可以叫作 `abc`，也可以叫做 `book`。也可以是字母和数字的组合，例如，`a1` 或者 `abc123` 都是可以的。当然也有一些限制，如果你想知道，请看附录A。

到这里，可能还有很多同学想问，int究竟是什么意思呢？

其实，`int` 控制小房子用来存放的数的类型，表示你目前申请的小房子只能存放整数。

`int` 是英文单词 `integer`（整数）的缩写。

如果要放小数该怎么办？



我们用 `float` 来申请一个小房子，用来存放小数，形式如下：

```
float a;
```

这样，小房子a就可以用来存放小数了，例如：

```
float a;
a=1.5;
printf("%f", a);
```

就表示申请一个用来存放小数的房子 `a`，里面存放了小数 `1.5`。

注意：在C语言中，小数成为浮点数，用float表示。

之前我们在用`printf`语句输出整数时，使用的是“%d”。此时需要输出小数，我们要用“%f”。

好了，我们来总结一下，这里的“小房子”在我们C语言的专业术语中称为变量。`int`和`float`说明小房子是用啦i存放何种类型的数，我们这里将其称为“变量类型”或者“数据类型”。

类似`int a;`或者`float a;`的这种形式，我们称作“定义变量”，他们的语法格式如下：

```
口语 [小房子的类型] [小房子的名称], [小房子的名称];
术语 [变量的类型] [变量的名称], [变量的名称];
代码 int a, b;
```

现在我们知道，`int a;`表示申请一个用来存放一个整数的小房子`a`，即定义一个整型变量`a`来存放整数；而`float a;`则表示申请一个用来存放一个小数的小房子`a`，即定义一个浮点数（实型）变量`a`来存放浮点数（小数）。

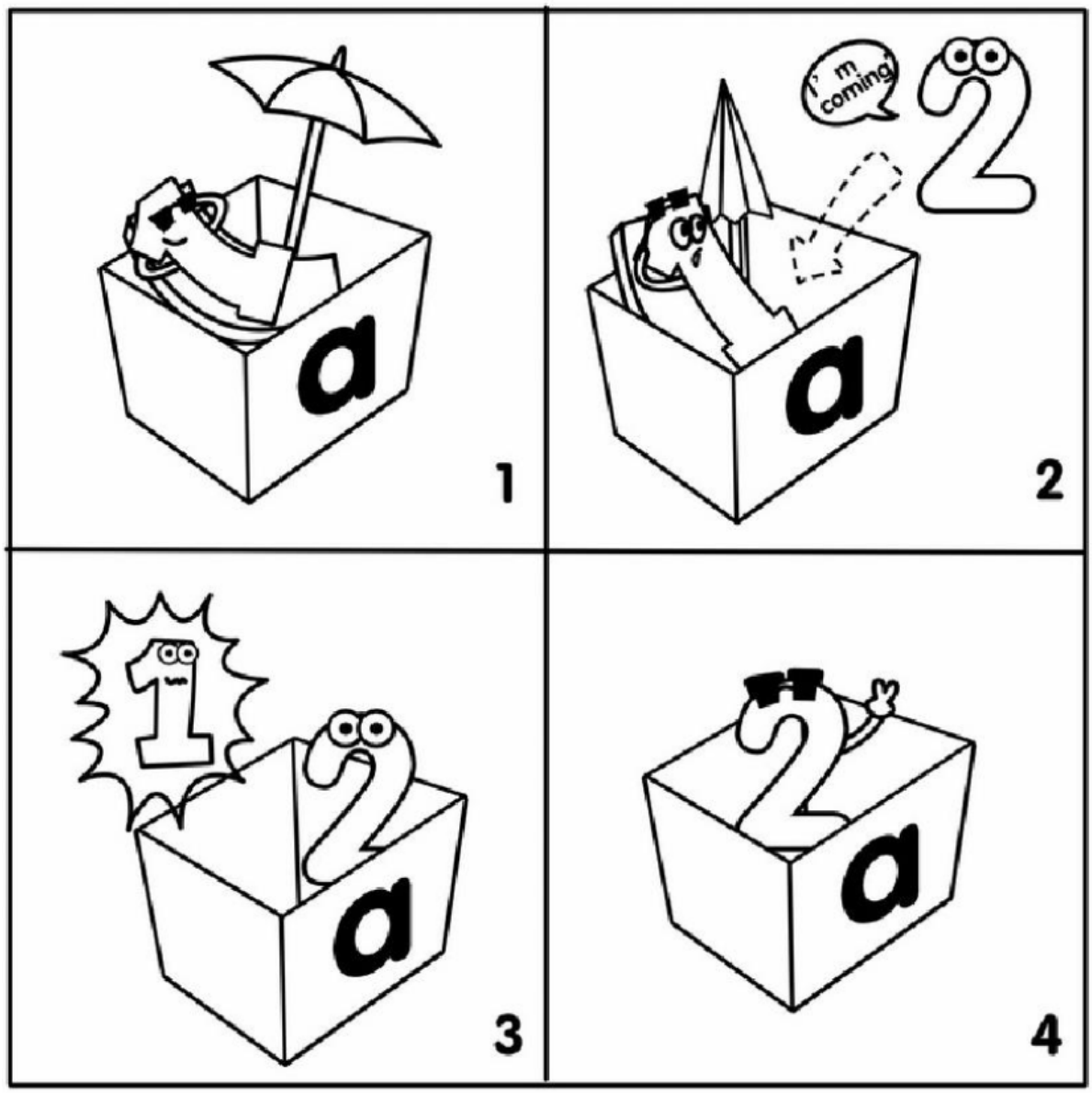
再来看另外一个有趣的问题，代码如下：

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a;
    a=1;
    a=2;
    printf("%d", a);

    system("pause");
    return 0;
}
```

请问计算机执行完上面的代码后，将会输出`1`还是`2`？



尝试过后你会发现，计算机显示的是 `2`，也就是说小房子 `a` 中的值最终为 `2`。通过观察代码我们可以发现，我们首先是将 `1` 放入小房子 `a` 中，紧接着又将 `2` 放入小房子 `a` 中，那么请问原来小房子中的 `1` 去哪里了呢？答案是被新来的 `2` 给覆盖了，原来的 `1` 已经消失了。也就是说，小房子 `a` 中有且仅能存放一个值，入股哦多次给小房子 `a` 赋值的话，小房子 `a` 中存放的将始终是最后一次赋的值。例如：

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a;
    a=1;
    a=2;
    a=3;
    a=4;
    a=5;
    a=6;
```

```
    printf("%d", a);

    system("pause");
    return 0;
}
```

计算机运行完上面这段代码后最终将输出6.也就是说小房子 `a` 中的值最终为6，前5此的赋值全部被覆盖掉了。

一个更有意思的问题来了，请继续看下面的代码：

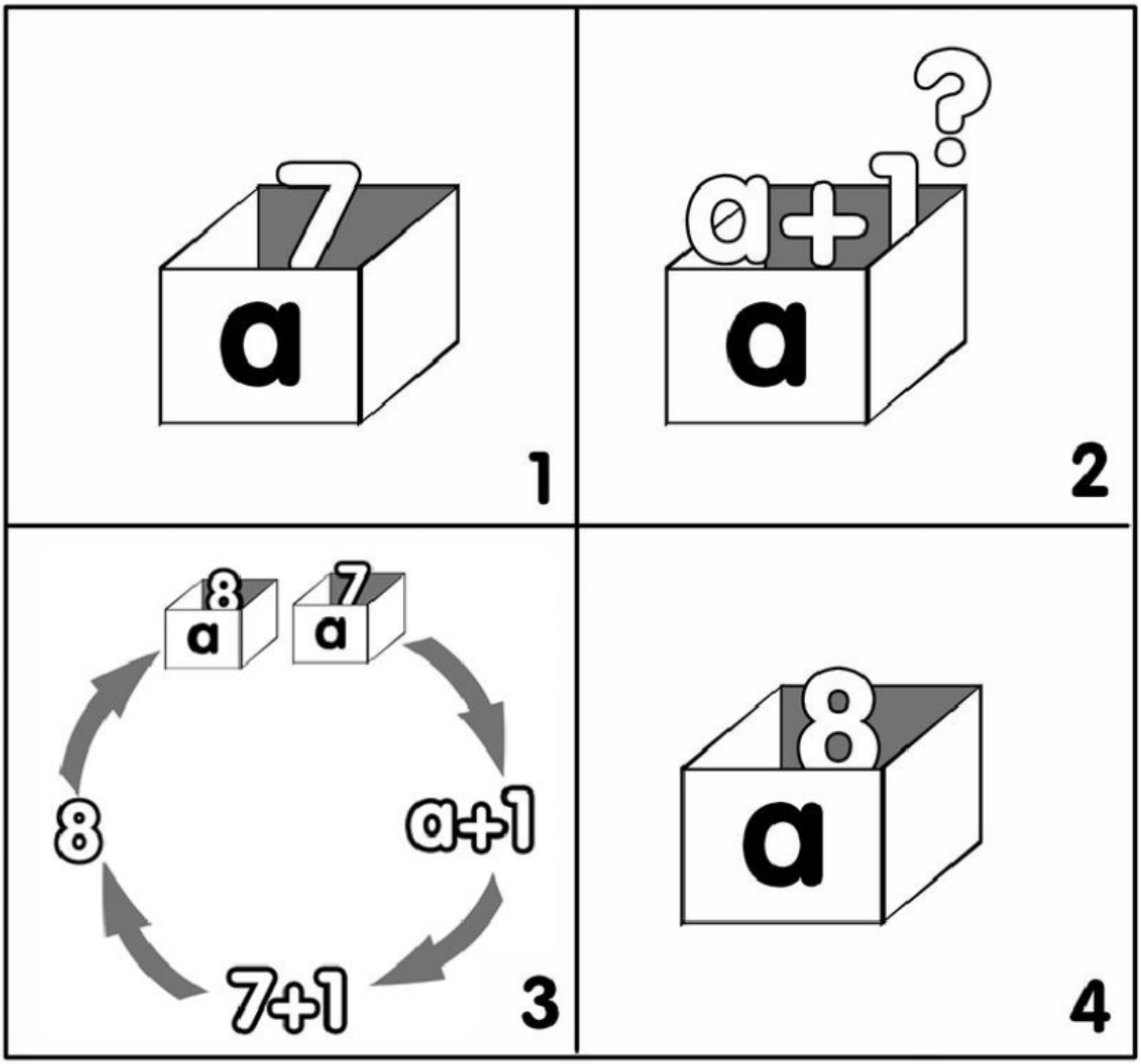
```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a;
    a=7;
    a=a+1;
    printf("%d", a);

    system("pause");
    return 0;
}
```

计算机运行完上面这段代码后最终将输出8.也就是说小房子 `a` 的之最终为8。计算机在执行完 `a=7;` 这句话后，小房子 `a` 中存储的值为7，之后计算机有紧接着运行了 `a=a+1;` 。运行完 `a=a+1;` 后，小房子 `a` 中的值就变为8了。也就是说 `a=a+1;` 的作用是把小房子 `a` 中的值在原来的基础上增加1，我们来分析一下这句话。

对于 `a=a+1;` 计算机分两步执行，这句话中有两个操作符，第一个是“+”，另一个是“=”（赋值符），应为“+”的优先级要比“=”高，因此计算机先执行 `a+1` ，此时小房子 `a` 中的值仍然为7，所以 `a+1` 的值为8.紧接着计算机就会执行赋值语句，将计算出来的值8再赋值给 `a` ，此时 `a` 的值就更新为8。



好啦，猜猜下面的程序，计算机最终会输出多少？

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a;
    a=10;
    a=a*a;
    printf("%d", a);

    system("pause");
    return 0;
}
```

尝试过来吗？想一想为什么a最终的值为100。

注：所有运算符的优先级详见附录B。

一起来找茬

1、下面这段代码是让计算机计算 `1.2x1.5` 的值。其中有5个错误，快来改正吧！

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a, b, c;
    a=1.2;
    b=1.5;
    c=a*b;
    printf("%d", c);

    system("pause");
    return 0;
}
```

更进一步，动手试一试

1、请进行两个小数的加法运算，例如：`5.2+3.1=?` 代码如下：

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    float a, b, c;
    a=5.2;
    b=3.1;
    c=a*b;
    printf("%f", c);

    system("pause");
    return 0;
}
```

请注意，之前我们在 `printf` 语句中输出整形变量的值时，使用的是“%d”，此时需要输出的是实型变量的值，我们要用“%f”

2、让计算机把下面3个公式算出来吧！

```
1.2+2.3+3.4+4.5
1.1*100
10.1*(10*10)
```

这一节，你学到了什么

1、如何定义一个用来存放小数的变量？

2、如何让一个小房子 `a` （变量a）中的值增加1？

第6节 数据输出——我说咋地就咋地

第7节 数据输入——我说算哈就算哈

第8节 究竟有多少种小房子

第9节 拨开云雾见月明

第10节 逻辑挑战1：交换小房子中的数

第11节 天啊！这怎么能看懂

第3章 较量才刚刚开始

- 第1节 大于、小于还是相等
- 第2节 判断正数
- 第3节 偶数判断
- 第4节 神器else
- 第5节 请告诉我谁大
- 第6节 逻辑挑战2：3个数怎么办
- 第7节 逻辑挑战3：我要排序
- 第8节 运算符总结
- 第9节 1>2究竟对不对

第10节 讨厌的嵌套

第11节 if-else语法总结

第4章 重量级选手登场

第1节 永不停止的哭声

第2节 我说几遍就几遍

第3节 if对while说：我对你很重要

第4节 求和！求和！求和！

第5节 逻辑挑战4：60秒倒计时开始

第6节 这个有点晕——循环嵌套来了

第7节 逻辑挑战5：奔跑的字母

第8节 究竟循环了多少次

第9节 逻辑挑战6：奔跑的小人

第10节 for隆重登场

第5章 好戏在后面

第1节 程序的3种结构

第2节 啰嗦一下

第3节 逻辑挑战7：判断质数很简单

第4节 更快一点：break

第5节 continue

第6节 逻辑挑战8：验证哥德巴赫猜想

第7节 逻辑挑战9：水仙花数

第8节 逻辑挑战10：解决奥数难题

第9节 逻辑挑战11：猜数游戏

第10节 逻辑挑战12：你好坏，关机啦

第6章 天啊！一大串数正在接近

第1节 逆序输出

第2节 申请100个小房子怎么办

第3节 100个数的逆序

第4节 逻辑挑战13：陶陶摘苹果

第5节 逻辑挑战14：一个萝卜一个坑

第6节 逻辑挑战15：选择排序

第7节 二维数组

第8节 剩下的一些东西

第7章 有了它你能做更多的事

第1节 字符的妙用

第2节 多余的回车键

第3节 字符的本质

第4节 人名怎么存储呢

第5节 逻辑挑战16：字母的排序

第6节 逻辑挑战17：字典序

✻ 第7节 多行字符

✻ 第8节 存储一个迷宫

第8章 游戏时间到了

✻ 第1节 走出迷宫

✻ 第2节 推箱子

附录A 标识符命名规则

标识符是用户编程时需要使用的名字。在日常生活中，我们指定某个东西、人，都要用到它、他或者她的名字。在编程语言中，对于变量、常量、函数也有名字，我们统称为标识符。我们在给人起名字时有一定的规矩，比如：头一个字为父亲或者母亲的姓氏，后面一般是一个或者两个子。在编程语言里的标识符也有一定的命名规则。

C语言的标识符分类三类：保留字（也称作关键词）、预定义标识符和用户标识符。C语言（C99）的保留字如下：

char	short	int	unsigned	long	float
double	struct	union	void	enum	signed
const	volatile	typeof	auto	register	static
extern	break	case	continue	default	do
else	for	goto	if	return	switch
while	sizeof	_Bool	_Complex	_Imaginary	inline
restrict					

预定义标识符是指C语言中有特定含义的标识符。例如函数`printf`、`scanf`、`sin`、`isalum`等，以及编译预处理命令名（如`define`、`include`）等。预定义标识符可以作为用户标识符使用，但是这样会失去系统规定的原意，我们不推荐这样做。

用户标识符一般是指用户自己定义的变量名和函数名等。用户在定义标识符时需要注意以下4点：

1、必须是字符（A-Z及a~z）、数字（0-9）和下划线的组合；

2、首字符不能是数字，但可以是字母或者下划线；

3、不能与保留字相同

4、标识符对大小写敏感，即严格区分大小写。

附录B 运算符的优先级和结合性